Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 2

Course: Operating Systems

Topic: Track B: Meet the OS You Will Build (xv6)

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 2 · Track B: Meet the OS You Will Build (xv6)

Environment & Toolchain

- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.8)
- **Target Architecture:** RISC-V 64-bit (`riscv64-linux-gnu`)
- **Emulator:** QEMU RISC-V System Emulator (`qemu-system-riscv`)
- **Toolchain Packages:** `git`, `base-devel`, `gdb`, `qemu-system-riscv`, `riscv64-linux-gnu-gcc`, `riscv64-linux-gnu-binutils`

Part 1. Build It and Boot It

1. Compiling and Launching xv6

To compile the kernel, bootstrap loader, and user-space utilities, and then launch them under QEMU:

```
❯ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
 ↪  -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device
 ↪  virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

2. Exiting QEMU

xv6 executes inside the QEMU hardware emulator. To quit and return to the host terminal:
- Press `Ctrl-A`, release, then press `X`.

Part 2. Use xv6 as the Unix It Is

Commands Tested Inside xv6

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 32800
echo           2 4 31696
forktest       2 5 15856
grep           2 6 36248
init           2 7 32176
kill           2 8 31568
ln             2 9 31448
ls             2 10 35056
mkdir          2 11 31664
rm             2 12 31648
sh             2 13 54088
stressfs       2 14 32544
usertests      2 15 178872
grind          2 16 47688
wc             2 17 33496
zombie         2 18 31032
```

```
logstress      2 19 33544
forphan        2 20 31736
dorphan        2 21 31600
sync           2 22 31104
console        3 23 0

$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
...

$ echo hello xv6
hello xv6

$ ls | grep c
cat            2 3 32800
wc             2 17 33496
sync           2 22 31104
console        3 23 0

$ wc README
47 314 2441 README
```

Observe (Part 2 Answers)

1. **List three programs xv6 ships with:**
   - `ls`: Lists directory entries with file type, inode number, and size.
   - `cat`: Concatenates and displays file contents on standard output.
   - `echo`: Prints its command-line arguments to standard output. *(Other utilities include `sh`, `grep`, `wc`, `rm`, `mkdir`, and `kill`).*
2. **You just used a pipe (`|`) inside xv6. Which two OS features must exist for a pipe to work?**
   - **Process Creation and Multiprocessing/Scheduling (`fork()`):** The operating system must be capable of creating, running, and context-switching two independent processes concurrently—a producer process (`ls`) and a consumer process (`grep`).
   - **Inter-Process Communication (IPC) via shared kernel buffers & file descriptor abstraction (`pipe()`):** The operating system must allocate a synchronized in-memory ring buffer in kernel space that binds the standard output file descriptor (`fd 1`) of the writer process to the standard input file descriptor (`fd 0`) of the reader process, handling flow control and blocking reads/writes.
3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**
   - While both shells interpret user commands and support fundamental Unix mechanisms like I/O redirection, pipes, and background execution (&), the xv6 shell is a minimalistic, bare-bones interpreter (~150 lines of C) without modern features such as command history, tab-completion, environment variables, scripting syntax, or job control.

Part 3. Read the Source

1. Repository Structure: The Kernel/User Split

The codebase cleanly separates privileged and unprivileged execution: - `kernel/`: The operating system itself. Runs in privileged RISC-V **Supervisor mode**. Implements memory virtualization (page tables), process management, context switching, interrupt/exception handling, the system call interface, and the file system. - `user/`: User-space applications (`cat`, `ls`, `echo`, `sh`, etc.). Runs in unprivileged **User mode**, isolated from direct hardware

access.

2. Tracing System Calls in `user/cat.c`

Inspecting `user/cat.c`:

```
void
cat(int fd)
{
  int n;
  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

System calls are declared in `user/user.h` and dispatched to kernel routines: - `user/user.h` declares function prototypes for user programs. - `user/usys.pl` generates assembly stubs (`usys.S`) that load the system call number into register `a7` and execute `ecall`. - `kernel/syscall.c` catches the trap, routes the call number, and calls the appropriate `sys_*` kernel handler.

Observe (Part 3 Answers)

1. **Which system calls does `user/cat.c` use, and what does each one ask the kernel to do?**
   - `open(argv[i], O_RDONLY)`: Requests the kernel to open the specified file path in read-only mode, allocating an entry in the process's file descriptor table and returning the integer file descriptor.
   - `read(fd, buf, sizeof(buf))`: Requests the kernel to read up to 512 bytes of data from the file or stream represented by `fd` into the user-space buffer `buf`.
   - `write(1, buf, n)`: Requests the kernel to transfer `n` bytes from the user buffer `buf` to file descriptor 1 (`stdout`).
   - `close(fd)`: Requests the kernel to close the open file descriptor `fd`, releasing the descriptor table entry and decrementing the file object reference count.
   - `exit(status)`: Requests the kernel to terminate the calling process, reclaim its virtual memory and open resources, and report its exit code to the parent process.
2. **In which kernel file, and on which line, is `sys_read` implemented?**
   - **File:** `kernel/sysfile.c`
   - **Line: 69**

   ```
   uint64
   sys_read(void)
   {
     struct file *f;
     int n;
     uint64 p;

     argaddr(1, &p);
     argint(2, &n);
     if (argfd(0, 0, &f) < 0)
       return -1;
     return fileread(f, p, n);
   }
   ```
3. **In one sentence: what is the difference between the code in `kernel/` and the code in `user/`?**

- Code in `kernel/` runs in privileged Supervisor mode with direct access to physical memory, hardware devices, page tables, and CPU control registers, whereas code in `user/` runs in unprivileged User mode with restricted address spaces and must invoke system calls (`ecall`) to request the kernel to perform privileged operations.

Part 4. Write Your First xv6 Program (`sleep`)

1. Compilation Error & Root Cause Analysis

When initially implementing `user/sleep.c` and building with `make qemu`, GCC produced the following error:

```
riscv64-linux-gnu-gcc ... -c -o user/sleep.o user/sleep.c
user/sleep.c: In function 'main':
user/sleep.c:11:3: error: implicit declaration of function 'sleep' [-Wimplicit-
function-declaration]
   11 |   sleep(atoi(argv[1])); // system call into the kernel
      |   ^~~~~
make: *** [<builtin>: user/sleep.o] Error 1
```

Root Cause: In this version of the course `xv6-riscv` repository: - The system call number is `#define SYS_pause 13` in `kernel/syscall.h`. - The kernel handler is `sys_pause()` in `kernel/sysproc.c`. - The user-space header `user/user.h` declares `int pause(int);`. - Because `-Werror` is enforced, invoking `sleep()` without a declaration triggers a fatal compile error.

Solution: Use pause

2. Implementation: `user/sleep.c`

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // system call into the kernel
  exit(0);
}
```

3. Adding `sleep` to `Makefile`

In `Makefile`, `$U/_sleep\` was added to the `UPROGS` list:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    $U/_pingpong\
    $U/_forktest\
    $U/_grep\
    ...
```

4. Verification in xv6

```
$ sleep 10
(pauses for 10 timer ticks, then returns to prompt)
$ sleep
usage: sleep <ticks>
```

Deliverables Checklist

- ☒ user/sleep.c: Working sleep program with argument validation and system call invocation.
- ☒ Makefile: Updated `UPROGS` with `_sleep`.
- ☒ notes.md: Answers to the Observe questions from Part 2 and Part 3.
- ☒ report.md: Full comprehensive laboratory report.